

Programación orientada a objetos (avanzada)

1º CFGS Desarrollo de Aplicaciones Web

Curso 2023/2024

Profesor: José Miguel Blázquez

Departamento de Informática

La clase ArrayList

- Estructura de datos dinámica: colección (grupo de objetos)
- Versión “dinámica” de la clase Array
- Implementa un array de tamaño variable:
 - Internamente usa un array tradicional, pero su crecimiento es dinámico
 - Eficiente para añadir al final y para buscar
 - NO eficiente para inserciones a mitad o borrados
- Indexación como arrays tradicionales (0 a N-1)
- Datos y objetos de cualquier tipo (operador < >)
- Necesita: `import java.util.ArrayList;`
- Requiere instanciar y emplear sus métodos

ArrayList

//Creación

```
ArrayList lista = new ArrayList(); // NO!!!  
ArrayList<String> lista = new ArrayList<String>();
```

//Insertar elementos (simples u objetos)

```
lista.add(-25);  
lista.add(new Persona("28751533Q", "María", "García", 37));
```

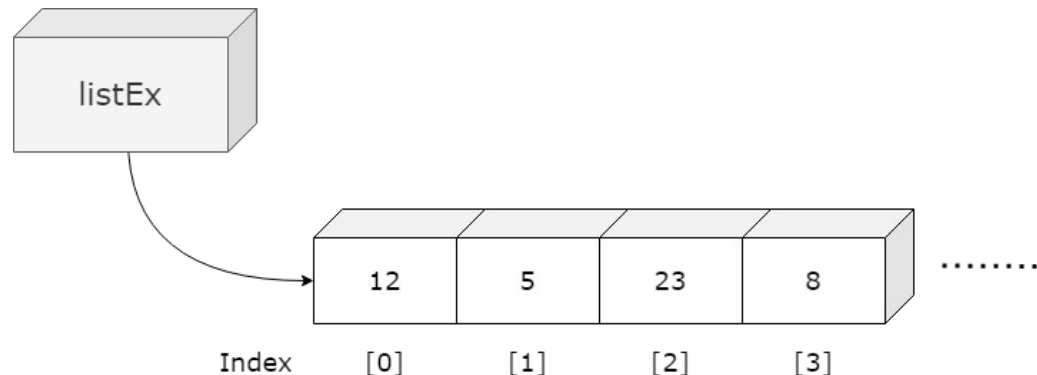
//o bien...

```
Persona p = new Persona("28751533Q", "María", "García", 37);  
lista.add(p);
```

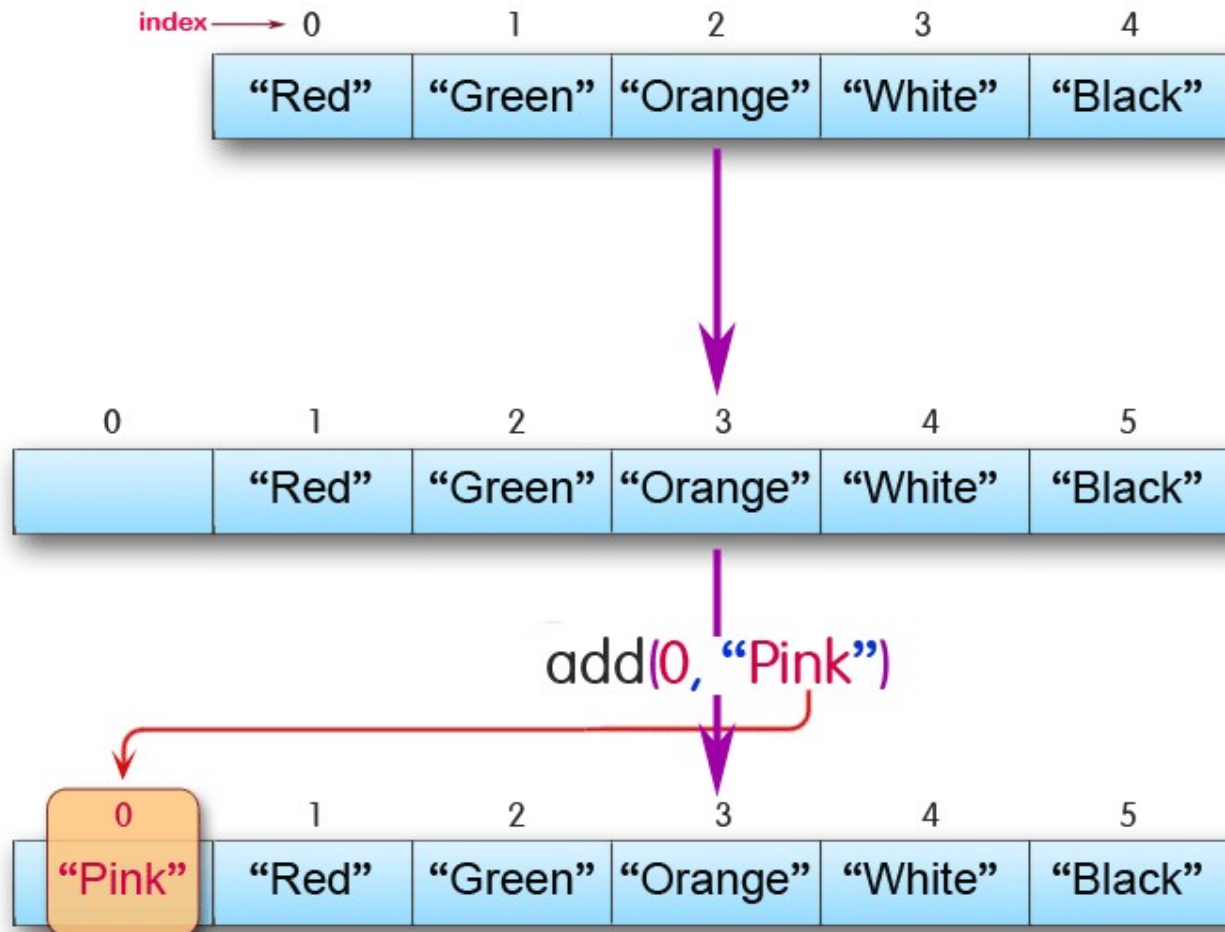
// Métodos destacados...ver API

add, get, size, remove(indice), indexOf, contains...

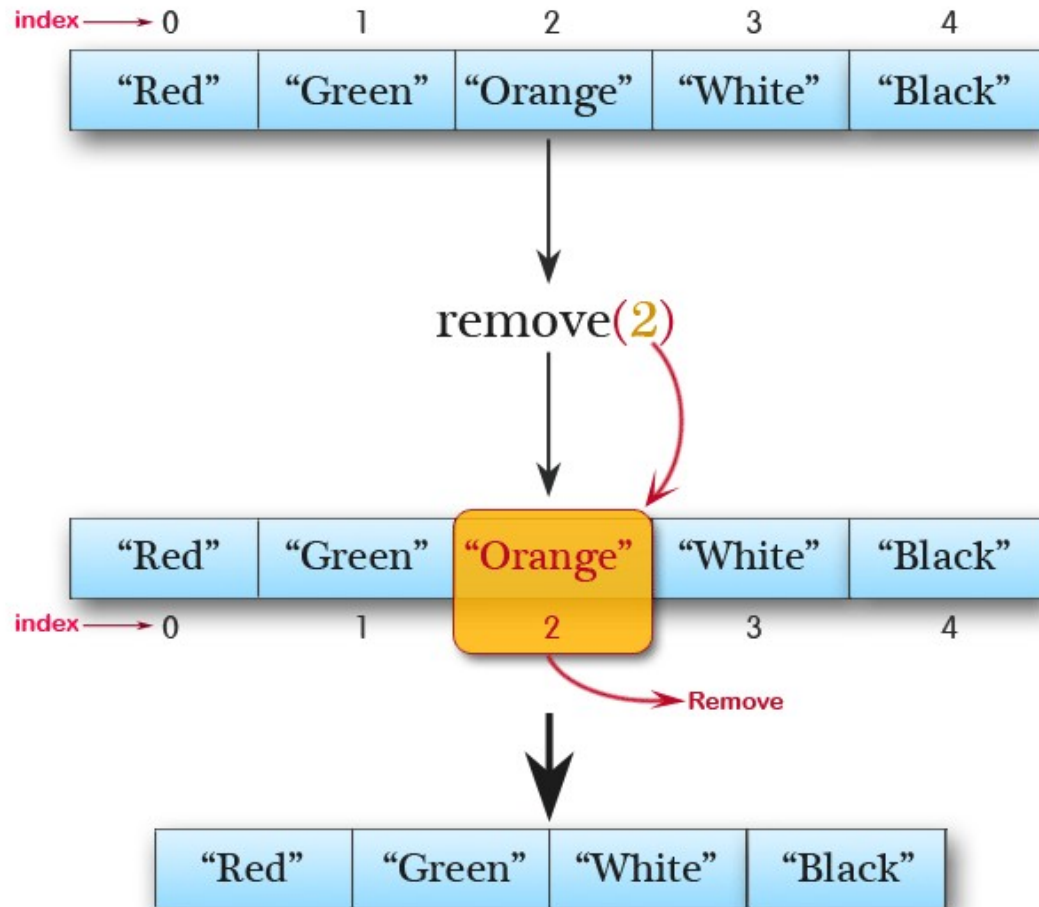
```
ArrayList<Integer> listEx = new ArrayList<>();  
listEx.add(12);  
listEx.add(5);  
listEx.add(23);  
listEx.add(8);
```



ArrayList



ArrayList



Recorrer un ArrayList

//3 formas:

```
for(int i = 0; i < lista.size(); i++) {  
    System.out.println(lista.get(i));  
}  
//-----  
for(String palabra: lista)  
    System.out.println(palabra);  
//-----  
Iterator iter = lista.iterator();  
while(iter.hasNext()) {  
    System.out.println(iter.next().toString());  
}
```

Eliminación de elementos durante la iteración:

No se debe utilizar `ArrayList.remove()` al iterar sobre elementos.
Hay que utilizar el método `Iterator.remove()`

ArrayList: Iterator

Particularidades del objeto Iterator

1. Si se realizan modificaciones directamente en la colección tras el Iterator, dará una excepción: **ConcurrentModificationException**

```
Iterator iter = lista.iterator();

lista.add(objeto1);
lista.add(objeto2);

while(iter.hasNext()) {
    System.out.println(iter.next());
}
```

¡Error!
Cambiar la creación del
Iterator y ponerlo tras
las llamadas a “add”.

2. Para iterar por una estructura con objetos, es importante instanciar un objeto de tipo auxiliar dentro del bucle, y también hacer un casting de tipos.
** Ver apuntes teóricos, apartado 1.7, donde hay un ejemplo de este punto.

ArrayList: ejemplos de métodos

// Declaración de un ArrayList de "String". Cualquier Elemento u Objeto (float...)

```
ArrayList<String> lista1 = new ArrayList<String>();
```

```
lista1.add("Elemento"); //Añade el elemento al ArrayList
```

```
lista1.add(n, "Elemento 2"); //Añade elemento en la posición 'n'
```

```
lista1.size(); //Devuelve numero de elementos del ArrayList
```

```
lista1.get(2); //Devuelve el elemento en la posición '2'
```

```
lista1.contains("Elemento"); //Comprueba si existe el elemento
```

```
lista1.indexOf("Elemento"); //Devuelve posición de 1ª ocurrencia  
'Elemento'
```

```
lista1.lastIndexOf("Elemento"); //Devuelve posición última  
ocurrencia 'Elemento'
```


ArrayList: métodos

```
lista1.remove(5); //Borra elemento de la posición '5'
```

```
lista1.remove("Elemento"); //Borra la 1ª ocurrencia de 'Elemento'
```

Cuidado, remove y otros causan una Excepción si llamamos dentro de un foreach. No sucede con un FOR normal o con un Iterator.

```
lista1.clear(); //Borra todos los elementos de ArrayList
```

```
lista1.isEmpty(); //Devuelve true si ArrayList vacío o false si no
```

```
ArrayList lista2 = (ArrayList) lista1.clone();  
//Copiar un ArrayList
```

```
Object[] array1 = lista1.toArray();  
//Pasa el ArrayList a un Array
```

Ejercicio rápido: modifica la implementación del programa que usaba la clase Persona, para que ahora se almacenen las 4 en un ArrayList.

Relaciones entre clases

- Programa: interacción entre objetos que se relacionan.
- Relaciones básicas:
 - Asociación (usa-conoce-se-relaciona-con)
 - Agregación (está-compuesto-por)
 - Composición (está-compuesto-por y lo necesita)
 - Herencia (es-un)
- Relaciones avanzadas:
 - Clases abstractas
 - Interfaces

Relación de asociación

```
public class Cliente{  
    private String nombre;  
    public CuentaCorriente cuenta;  
}  
  
public class CuentaCorriente{  
    private double saldo;  
    public Cliente titular;  
}
```

Bidireccional con cardinalidad 0..1 ó 1.

¿Y si no quiero bidireccional?

¿Y si quiero aumentar la cardinalidad a N?

- Cardinalidad
- Navegabilidad
- Depende de la lógica que demos y las restricciones que impongan los requisitos del software.

Relación de asociación



```
public class Cliente{
    private String nombre;
    private CtaCte cuenta;

    public Cliente(String n){
        nombre=n;
    }

    public void agregarCuenta(CtaCte c){
        cuenta=c;
    }
}
```

```
public class CtaCte{
    private double saldo;
    private Cliente dueño;

    public CtaCte(double s, Cliente d){
        saldo=s;
        dueño=d;
    }
}
```

Relación de asociación



```
public class Persona{
    private String nombre;
    private ArrayList<Perro> mascotas = new ArrayList();

    public Persona(String n){
        nombre=n;
    }

    public void agregarMascota(Perro p){
        mascotas.add(p);
    }
}
```

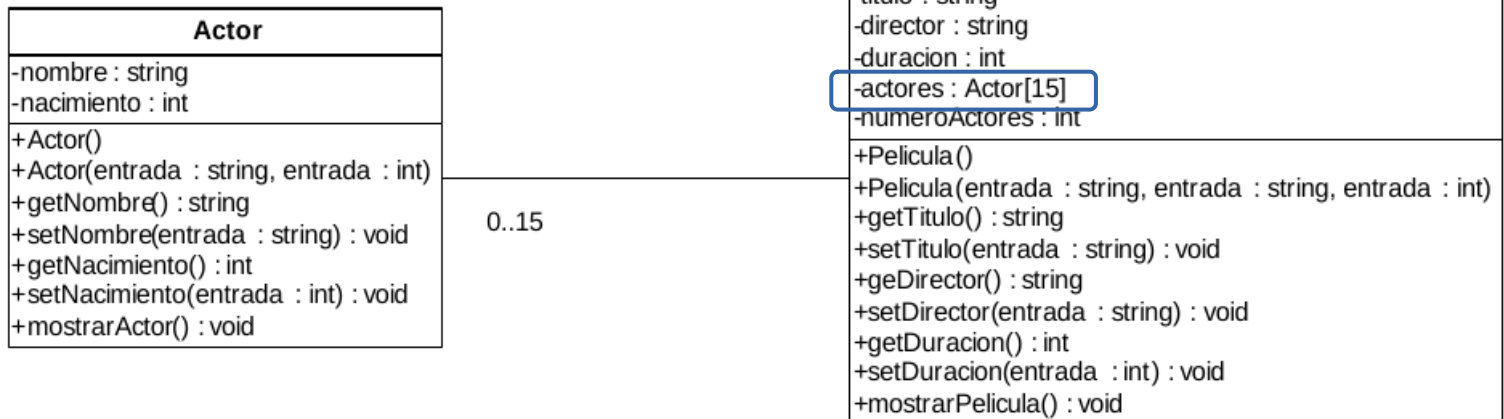
```
public class Perro{
    private String nombre;
    private Persona propietario;

    public Perro(String n){
        nombre=n;
    }

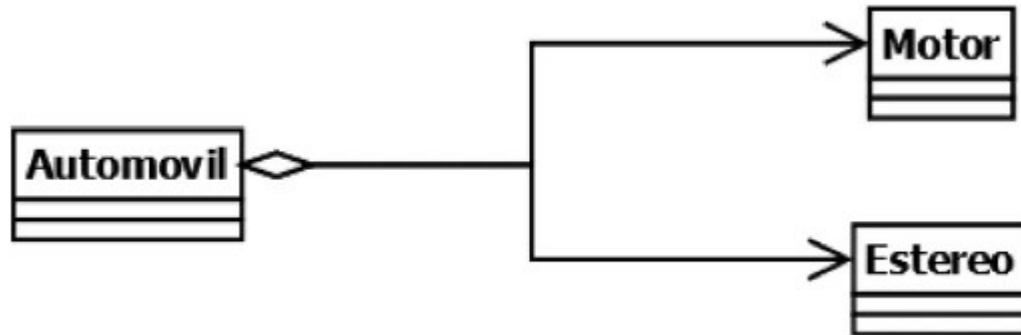
    public void asignarPropietario(Persona p){
        propietario=p;
    }
}
```

Relación de agregación

- Ejemplo clásico: Empresa – Clientes
- Una clase es parte de otra clase (composición débil).
- Unidireccionales. La destrucción del compuesto no conlleva la destrucción de los componentes.
- Implementación: referencia simple...
- Otro ejemplo: Actor – Película



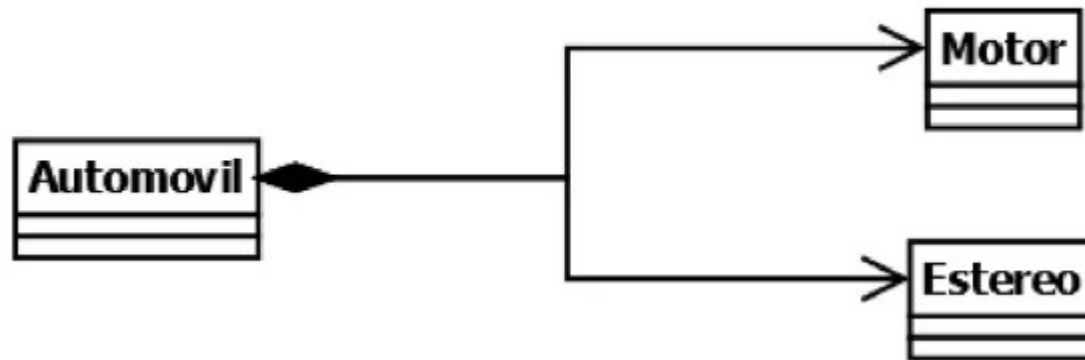
Relación de agregación



```
public class Automovil{
    private Estero estereo;
    private Motor motor;

    public void ensamblar(Estereo e, Motor m){
        estereo=e;
        motor=m;
    }
}
```

Relación de composición

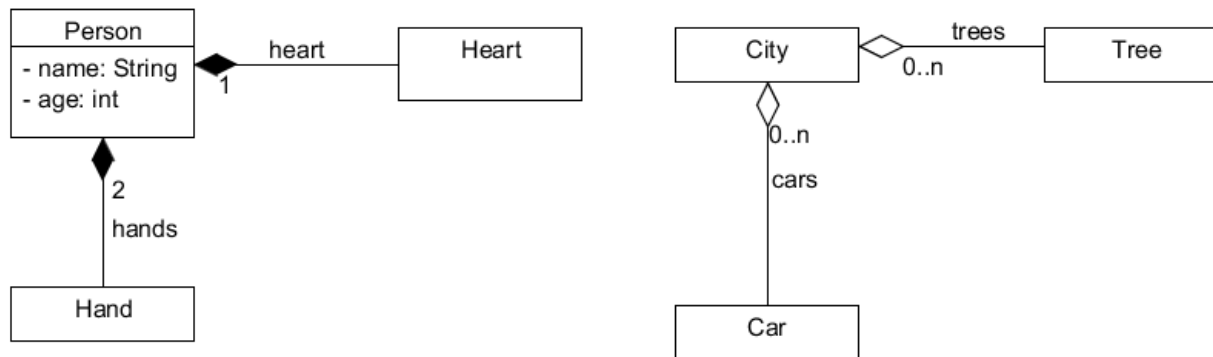


```
public class Automovil{
    private Estereo estereo;
    private Motor motor;

    public Automovil(){
        estereo = new Estereo();
        motor = new Motor();
    }
}
```

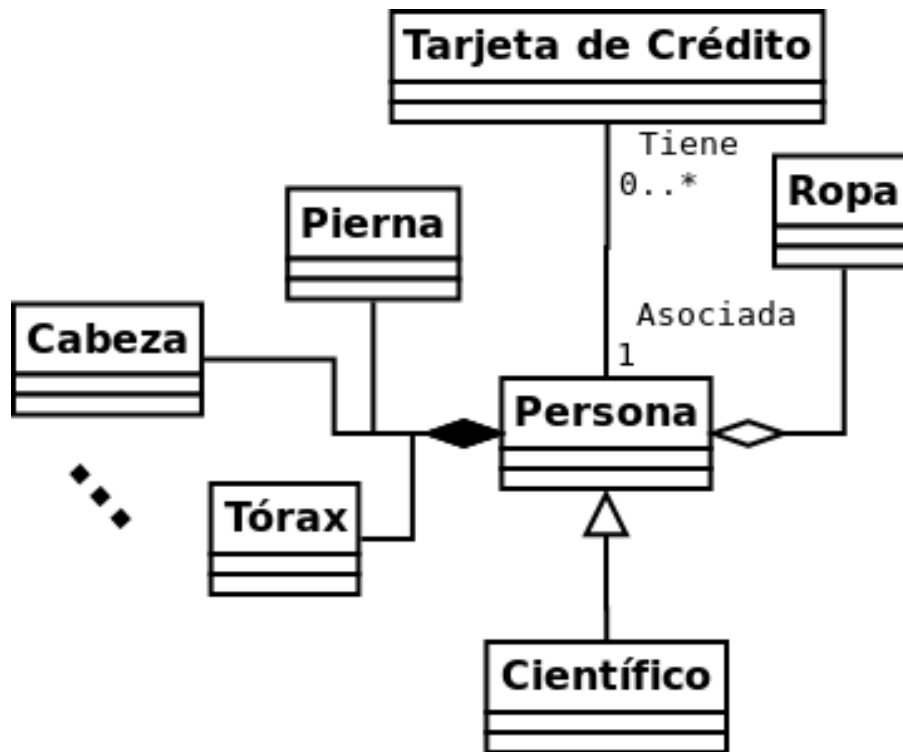

Composición vs Agregación

- Componentes constituyen una parte del objeto compuesto. Supresión del objeto compuesto conlleva supresión de componentes. Alto grado de dependencia.
- Implementación: referencias...pero, depende del contexto. Aspectos de diseño (ejemplo de sentencia ON DELETE... de SQL).



- **Composición** (Persona, Corazón, Mano), los "subobjetos" (Corazón, Mano) serán destruidos si la Persona es destruida. Ambigüedades conceptuales: ¿qué pasa con Biblioteca-Libro?: ¿Agregación o composición?
- **Agregación** (Ciudad, Árbol, Coche) "subobjetos" (Árbol, Coche) NO serán destruidos si la Ciudad es destruida.

Resumen de relaciones



- **Composición:** entre la clase Persona y las clases Pierna, Cabeza y Tórax entre otras tantas posibles).
- **Agregación:** Persona y Ropa).
- **Asociación:** Persona y Tarjeta de Crédito).
- **Herencia** (entre las clases Persona y Científico, la estudiaremos más adelante).

Recuerda: la composición es muy similar a la agregación. con la única diferencia de su objetivo de enfatizar la dependencia de la clase contenida al ciclo de vida de la clase contenedora. Es decir, la clase contenida será borrada si la clase contenedora es destruida.

Resumen de implementación

En general:

- Si no se puede acceder al atributo interno que forma la relación desde fuera de la clase contenedora, entonces tenemos una **composición**.
- De lo contrario, tenemos una **agregación**.

¡Agregación!

```
class Coche {  
    private final Motor mot;  
  
    public Coche(params){  
        mot = new Motor(params);  
    }  
}
```

¡Composición!

```
class Coche {  
    private Motor mot;  
  
    public Coche(Motor motExterno) {  
        this.mot = motExterno;  
    }  
  
    public Motor getMotor() {  
        return mot;  
    }  
}
```

Composición: el motor está completamente encapsulado por el Coche. No hay forma de que el mundo exterior obtenga una referencia al Motor.

Agregación: también hay un motor, pero puede ser cambiado o eliminado. El mundo exterior podría modificarlo sin importar si está en el Coche.

Implementación de relaciones II

Agregación

```
public class Departamento{
    private String nombreDepartamento;
    private ArrayList<Profesor> listaProfesores;

    //Los profesores ya existen y recibo el listado por parámetro
    public Departamento(ArrayList<Profesor> listadoProfesores){
        this.listaProfesores = listaProfesores;
    }
}
```

Los Profesores no desaparecen porque desaparezca el objeto Departamento

Composición

```
public class Universidad{
    private String nombreUniversidad;
    private ArrayList<Departamento> listaDeptos;

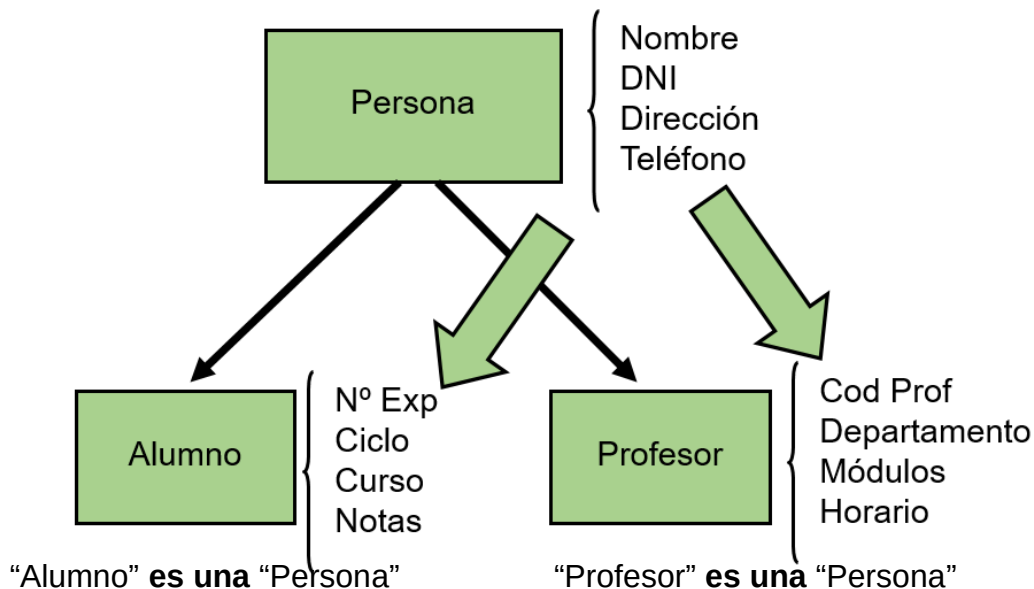
    public Universidad(){
        this.listaDeptos = new ArrayList<>();
    }

    public void nuevoDepto(String codigo, String nombreDepto){
        Departamento nuevoDepto = new Departamento(codigo, nombreDepto);
        listaDeptos.add(nuevoDepto);
    }
}
```

Si desaparece el objeto Universidad desaparecen también los departamentos que posee

Herencia

- Derivar o extender una clase a partir de otra, que comparte cosas comunes.
- Superclase (clase padre) y subclase (clase hija).
- Subclase hereda atributos y métodos del padre (depende de la visibilidad, privado no).
- Tipos:
 - Simple: una clase sólo hereda de otra. Java, C#. Es lo habitual y recomendado.
 - Múltiple: una clase puede heredar de varias. C++. Más complicada y peligrosa.



- Atributos comunes vs específicos.
- ¿esquema del dibujo sin herencia?

Herencia

- “extends”: en Java indica que se hereda de...
- Constructor: llamada “super()” obligatoria
 - Original o por defecto
- Acceso **directo** a miembros clase padre:
 - Private: imposible (sólo a través de getters/setters).
 - **Protected**: sí, sólo desde clase heredada.
 - Public: siempre.
- Buena práctica: usa private, no public, y obliga a usar getters y setters.

Herencia

```
public class Persona{  
    protected String nombre;  
    protected String dni;  
    protected String direccion;  
    protected String teléfono;  
}
```

Visibilidad
“protected” en
clase padre

```
public class Alumno extends Persona {  
    private String numexp;  
    private String ciclo;  
    private String curso;  
    private double[] notas;
```

“extends”
genera las
clases hijas,
sólo hay que
definir atributos
adicionales

```
    public Alumno(String nombre, String dni...String numexp...) {  
        super(nombre, dni...);  
    }
```

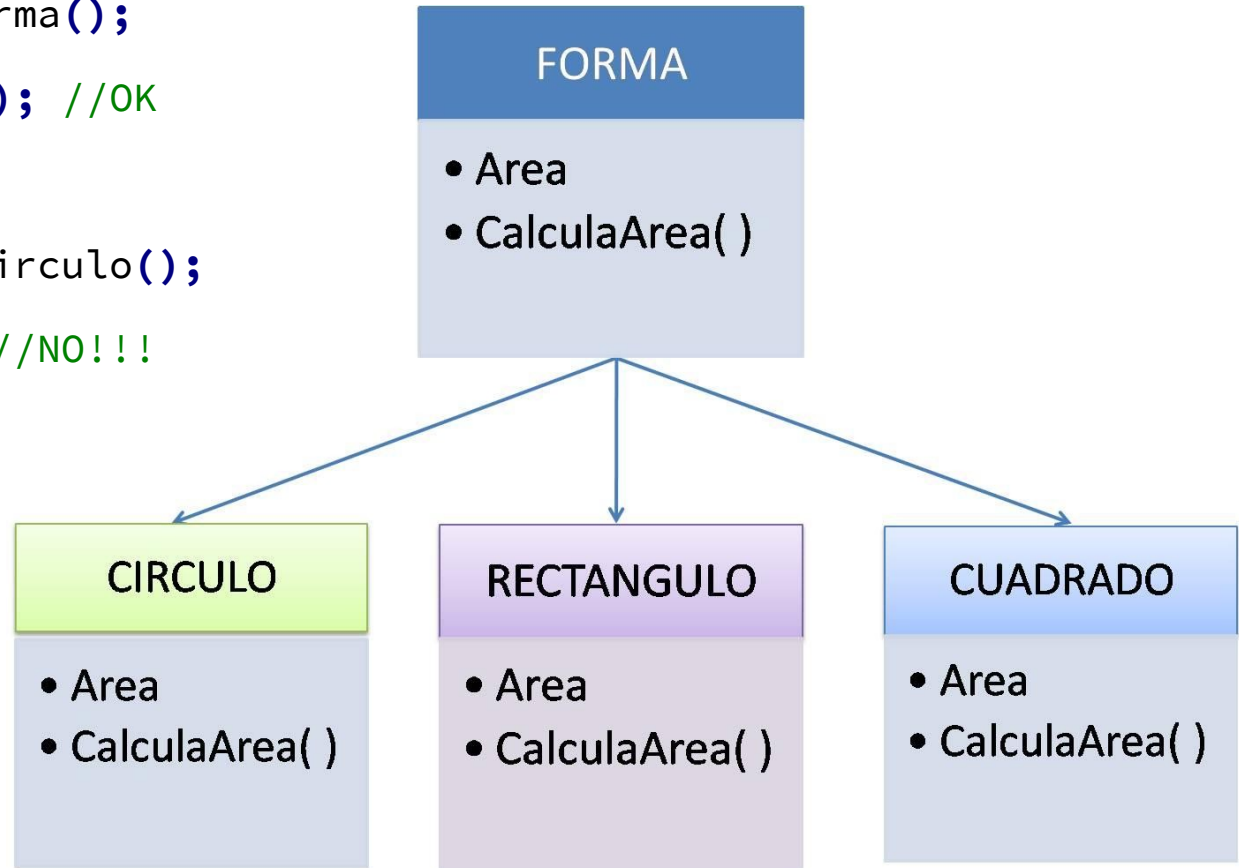
```
public class Profesor extends Persona {  
    private String codprof;  
    private String departamento;  
    private String[] modulos;  
    private Horario horario;  
}
```

Polimorfismo

- Capacidad de los objetos de ofrecer respuestas diferentes según el contexto.
- Concepto ligado a la herencia.
- Tipos:
 - Sobrecarga (polimorfismo paramétrico o ad-hoc): métodos con el mismo nombre pero se usan diferentes parámetros. Visto en constructores...puede existir sin herencia.
 - Sobrescritura (polimorfismo de inclusión): métodos con el mismo nombre en clases diferentes (generalmente tras herencia), requiriendo redefinición de su comportamiento.
 - Variables polimórficas: pueden comportarse, según contexto, como una clase o como una de sus subclases.

Polimorfismo

```
Forma fo1 = new Forma();  
fo1 = new Circulo(); //OK  
//-----  
Circulo c1 = new Circulo();  
c1 = new Forma(); //NO!!!
```



```
float CalculaArea(){  
    area = 3.1416*Math.pow(radio,2);  
}
```

Son iguales...conclusión?

```
float CalculaArea(){  
    area = base*altura;  
}
```

```
float CalculaArea(){  
    area = base*altura;  
}
```

Sobrescritura

```
class Animal {  
    public void hablar() {  
        System.out.println("Grr...");  
    }  
}  
class Gato extends Animal {  
    @Override  
    public void hablar() {  
        System.out.println("Miau.");  
    }  
}  
class Perro extends Animal {  
    @Override  
    public void hablar() {  
        System.out.println("Guau.");  
    }  
}
```

```
public static void main() {  
    Animal p = new Perro();  
    Animal g = new Gato();  
}  
  
p.hablar();  
//Salida "Guau."  
  
g.hablar();  
//Salida "Miau."
```

Si "Perro" no hubiera implementado (sobrescrito) el método "hablar", ¿podría usarlo?

Sobrecarga

- Puede existir sin herencia, dentro de una misma clase (constructores)

```
class Persona{  
    private String nombre;  
    private int edad;  
  
    public Persona(String name){  
        nombre = name;  
        edad = 0;  
    }  
  
    public Persona(String name, int age){  
        nombre = name;  
        edad = age;  
    }  
  
    public Persona(){  
        nombre = "Anónimo";  
        edad = 0;  
    }  
}
```

3 constructores diferentes...

En la creación del objeto,
según los parámetros, se
llamará a uno u otro
método...

Sobrecarga

Conocemos un uso de la sobrecarga desde inicio de curso:

```
System.out.println();
```

```
// Llamadas de ejemplo
```

```
[...].println(5);
```

```
[...].println("Hola...");
```

```
// Estructuras diferentes de println
```

```
// Definidas en System.out
```

```
System.out.println(int num);
```

```
System.out.println(String cad);
```

```
//Etcétera...
```

Variables polimórficas

```
Animal a = new Animal();  
a.hablar(); // Dirá "Grr..."
```

```
a = new Perro();  
a.hablar(); // Dirá "Guau."
```

//Podemos tener diversos subtipos en un array del supertipo

```
Animal[] animales = new Animal[2];  
animales[0] = new Perro();  
animales[1] = new Gato();
```

//Sin embargo, no se permite almacenar un objeto de la superclase en una variable de una subclase...

```
Perro p = new Animal();  
//Error de compilación
```

Clases abstractas

- Declara existencia pero no implementación.
- Métodos abstractos (min 1) y no abstractos.
- Sólo permiten herencia, no instanciación (no puedo hacer ***new*** sobre una abstracta).
- Uso: definir aspectos comunes a muchas clases.
- Subclases derivadas DEBEN implementar obligatoriamente los métodos abstractos, si no, da error.

Clases abstractas

```
public abstract class Principal {  
    // Método concreto con implementación  
    public void metodoConcreto(int n) {  
        System.println(n);  
    }  
    // Método abstracto sin implementación  
    public abstract void metodoAbstracto();  
}  
  
public class Secundaria extends Principal {  
    // Implementación concreta y obligatoria  
    public void metodoAbstracto() {  
        System.out.println("Método implementado.");  
    }  
}  
  
Principal p1 = new Principal(); //NO!!! es abstracta!  
Secundaria s1 = new Secundaria(); //SI!!
```

Clases abstractas: ejemplo

```
public abstract class Vehiculo{
    //atributos...
    boolean arrancado = false;

    //método normal
    public boolean arrancarParar(){
        this.arrancado = !this.arrancado;
    }
    //métodos abstractos
    public abstract void acelerar(int intensidad);

    public abstract void frenar(int intensidad);
}
```

```
Vehiculo miVehiculo;

miVehiculo = new Vehiculo() // Error

//Pero puede contener objetos de subclases
miVehiculo = new Coche();
miVehiculo.arrancarParar();
```

Polimorfismo

- Heredar para generar vehículos: coche, moto, camión...
- Cada una tiene su cálculo específico de la aceleración y freno
- Las subclases siempre van a sobrescribir esos métodos
- No existen en la realidad vehículos, sino subtipos.

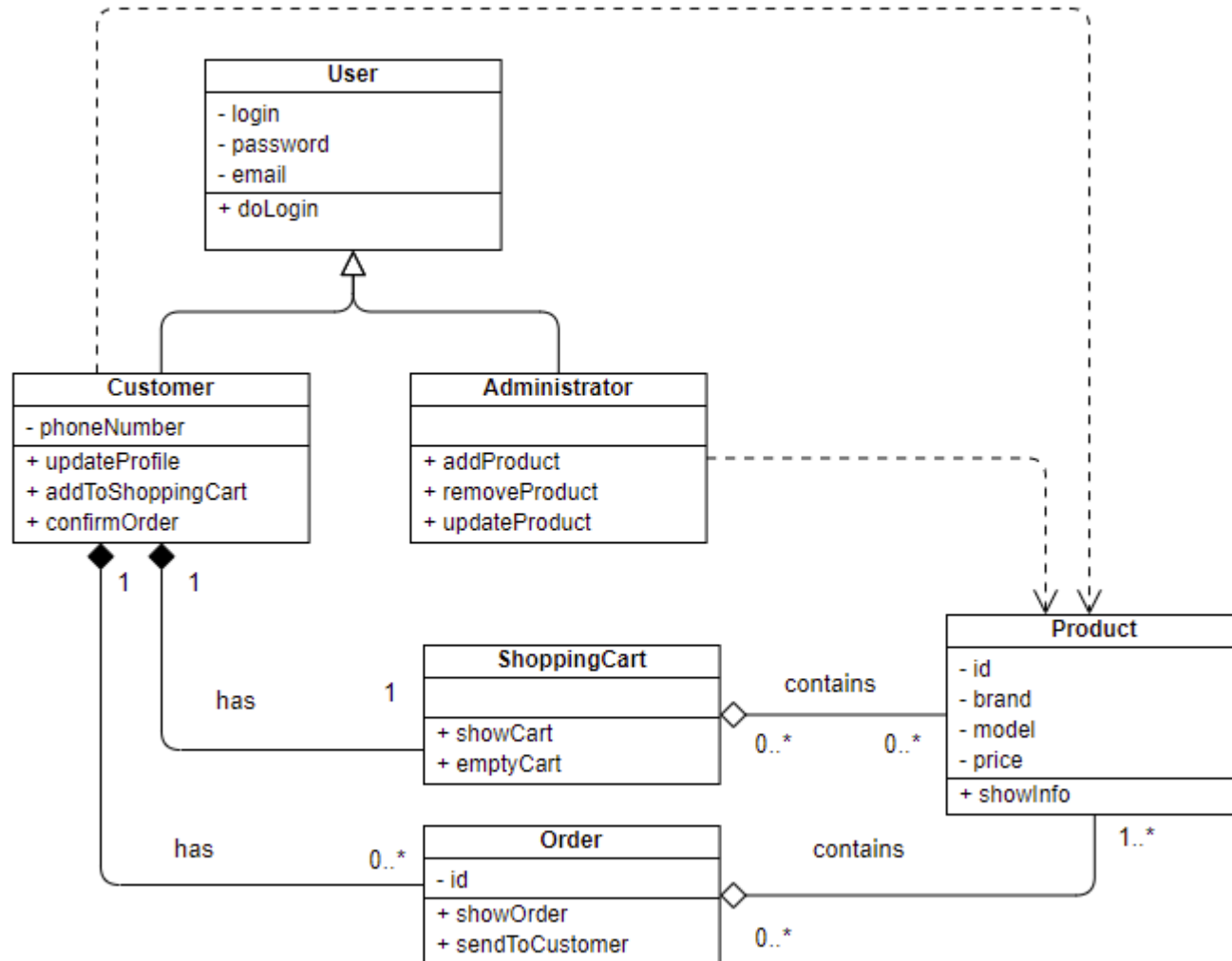
Clases abstractas: ejemplo

```
public class Coche extends Vehiculo {  
  
    private int puertas;  
    private int caballos;  
  
    public Coche(int cv, int puertas) {  
        caballos = cv;  
        this.puertas = puertas;  
    }  
  
    public void acelerar() {  
        // implementar  
    }  
  
    public void frenar() {  
        // implementar  
    }  
}
```

Vehiculo es abstracta
porque...

¿hay “vehículos” tangibles
o todo son subtipos?

Ejemplo completo relaciones



Interfaces

- Resolver limitaciones herencia simple.
- Definir roles genéricos a clases sin relación.
- Interfaces ~ clases abstractas, pero no implementan nada.
- Una clase puede implementar varias interfaces: así se consigue simular la herencia múltiple que Java no permite.
- Permiten la existencia de variables polimórficas y la invocación polimórfica de métodos.
- Relacionar clases que pertenecen a jerarquías de herencia diferentes.

Interfaces

- Características destacadas:
 - Todos sus elementos son `public`.
 - Todos sus métodos son abstractos.
 - Todos sus campos son `final static`.
- Sin tener que indicarlo explícitamente.

Interfaces

```
public interface VehiculoAereo{  
    void despegar(float alturaBase);  
    void aterrizar();  
}
```

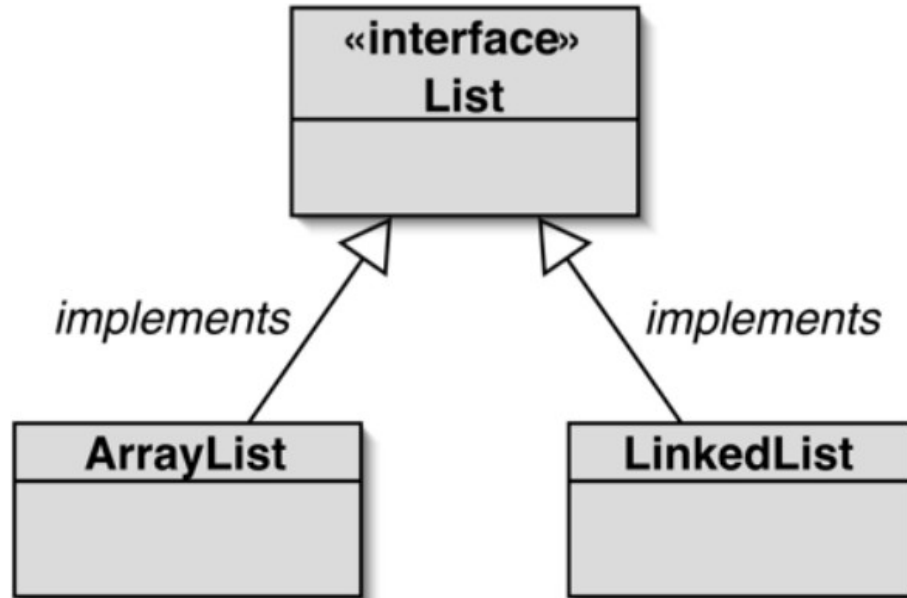
```
public class Globo extends Vehiculo implements VehiculoAereo{  
    private int volumenGlobo;  
    private int capacidadPasajeros;  
  
    //Constructores, getters, setters  
    @Override  
    void despegar(float alturaBase){  
        if(alturaBase > 500){  
            aumentarCombustible();  
        }  
    }  
  
    @Override  
    void aterrizar(){  
        reducirVelocidadVertical();  
    }  
}
```



Puede implementar
varias separadas
por comas

Podrían haber otros
“VehiculoAereo” que
no participaran en una
relación con Globo.

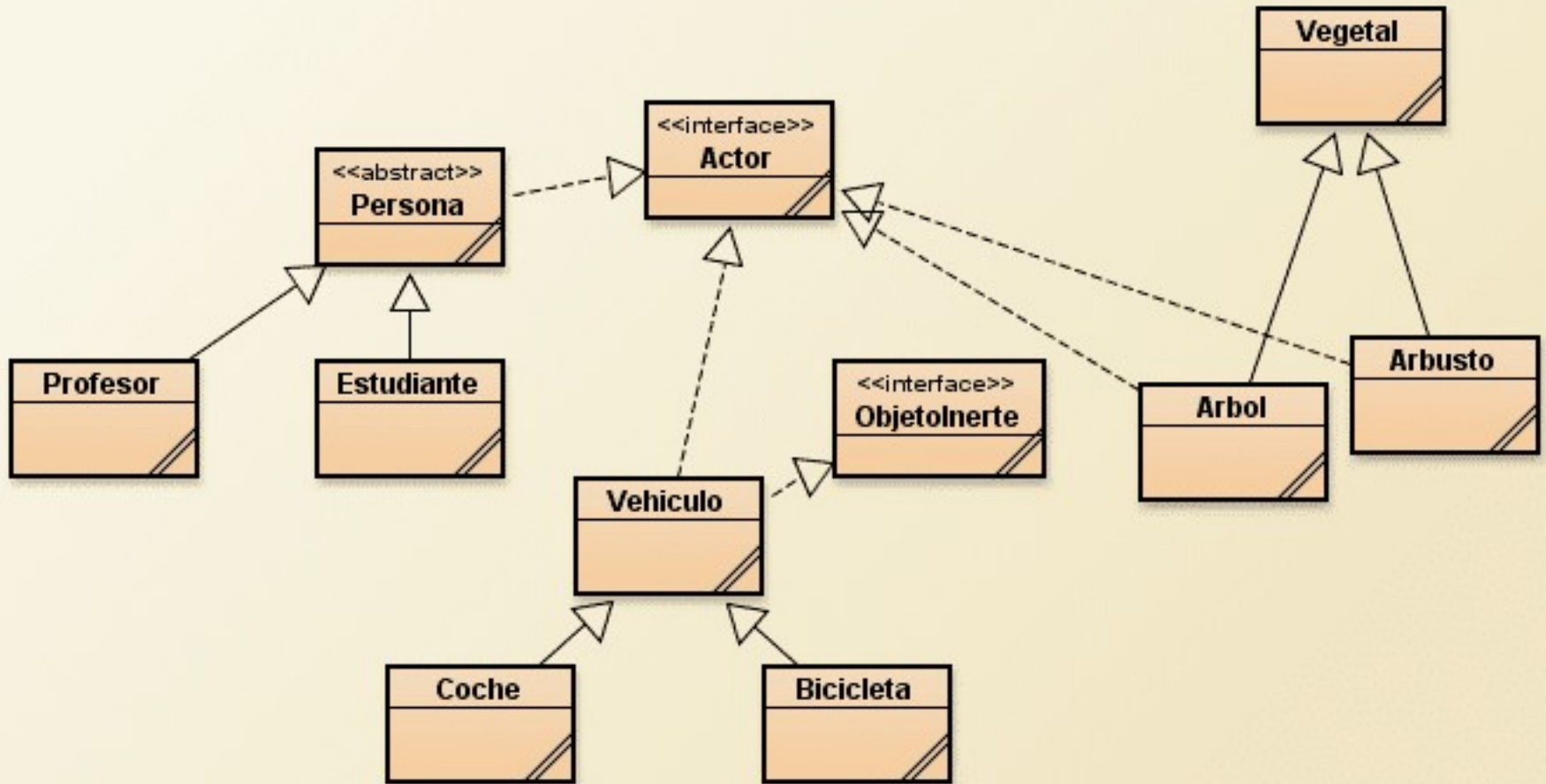
Interfaces



```
List<String> lista1 = new ArrayList<>();  
List<String> lista2 = new LinkedList<>();  
  
lista1.add("texto");  
lista2.add("texto");
```

Permite el
polimorfismo
sin la herencia
pura.

Interfaces



Ordenación de objetos

```
class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad){  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    //etcétera  
}  
  
public static void main(String[] args){  
    Persona[] personas = new Persona[50];  
  
    personas[0] = new Persona("Ana", 30);  
    personas[1] = new Persona("Paco", 40);  
    //etcétera  
}
```

```
class Persona implements Comparable<Persona>{  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad){  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    // Setters y getters  
  
    @Override  
    public int compareTo(Persona p){  
        if (this.getEdad() > p.getEdad())  
            return -1;  
        else if (p.getEdad() > this.getEdad())  
            return 1;  
        else  
            return 0;  
    }  
}
```

Collections.sort()
si usas un ArrayList

```
import java.util.Arrays;  
//-----  
public static void main(String[] args){  
    Persona[] personas = new Persona[50];  
    ... // Llenar el array con 50 personas reales  
    Arrays.sort(personas);  
    // Aquí el array ha quedado ordenado por edad (descendente)  
}
```